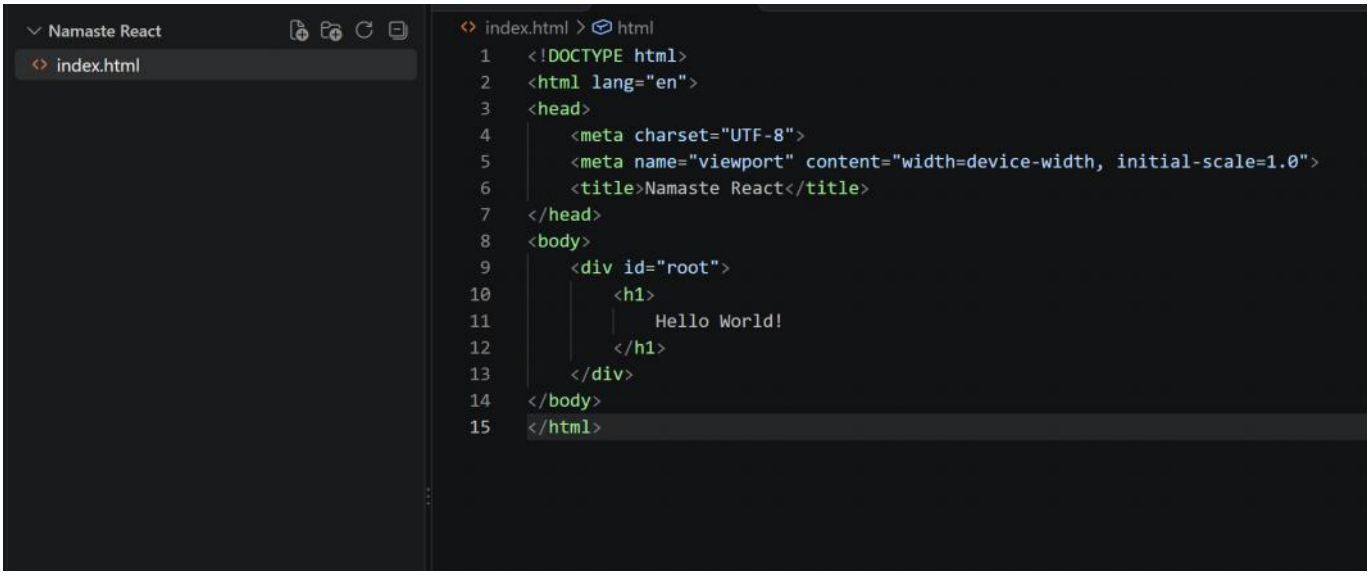
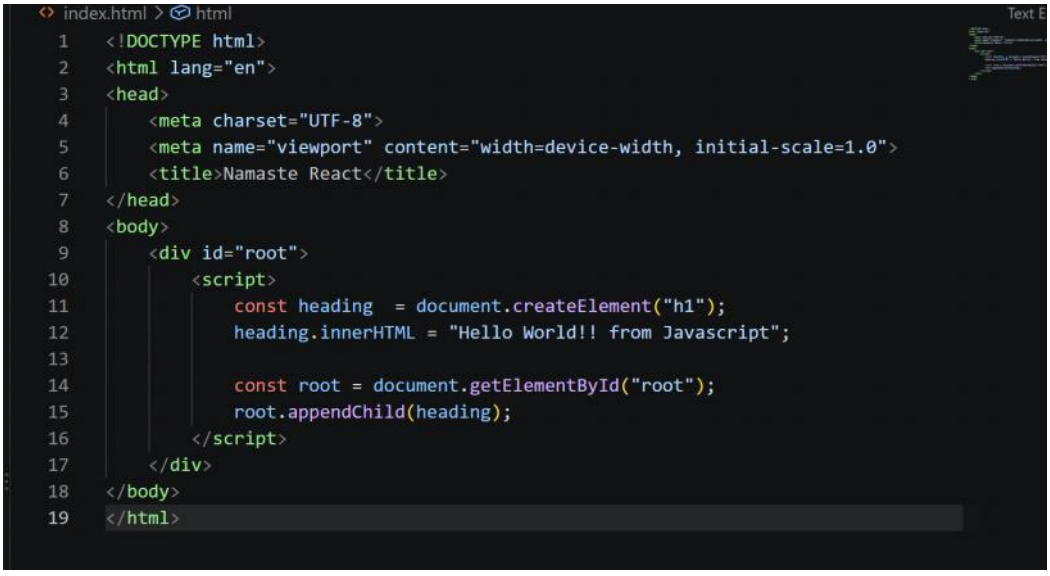
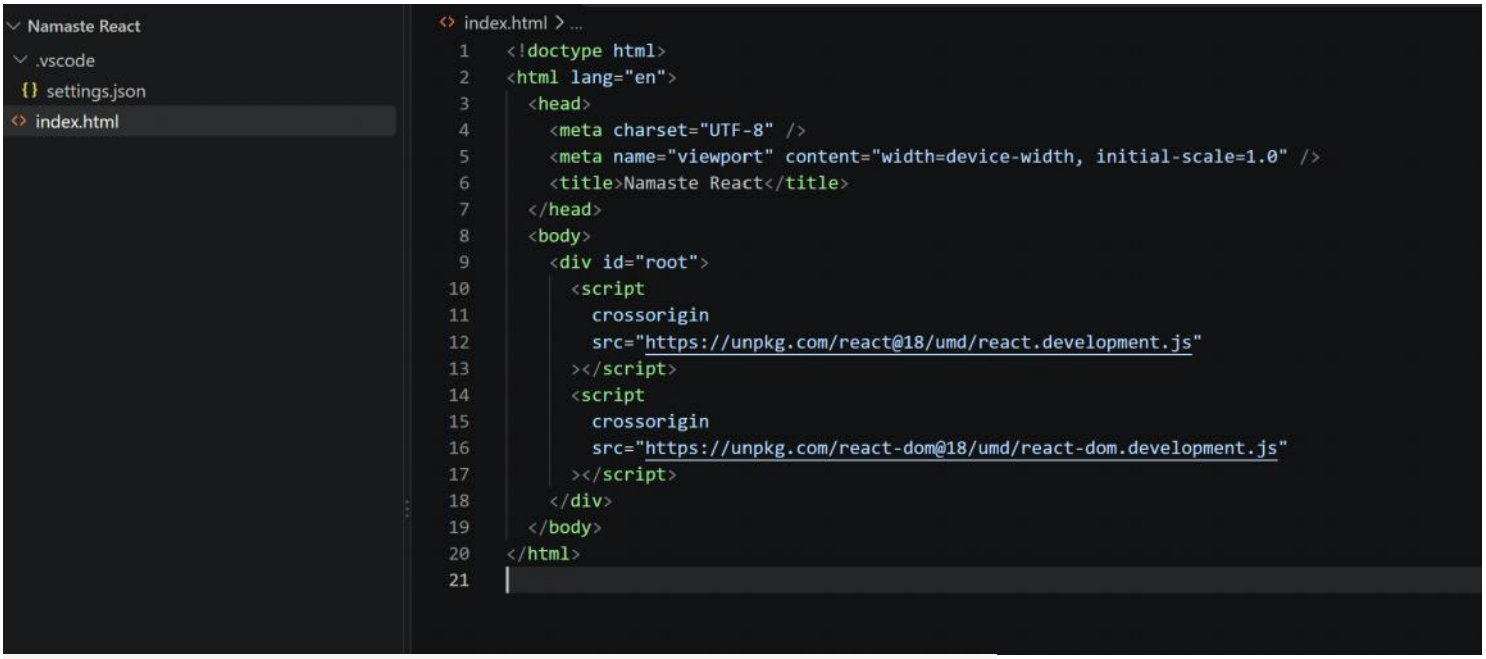
Begin with Html and Javascript Then React Super Basic

Tuesday, October 13, 2026 5:08 PM







This tells the browser: “Load this script from another origin, but don’t send credentials, and allow detailed error reporting.”

What is CDN?

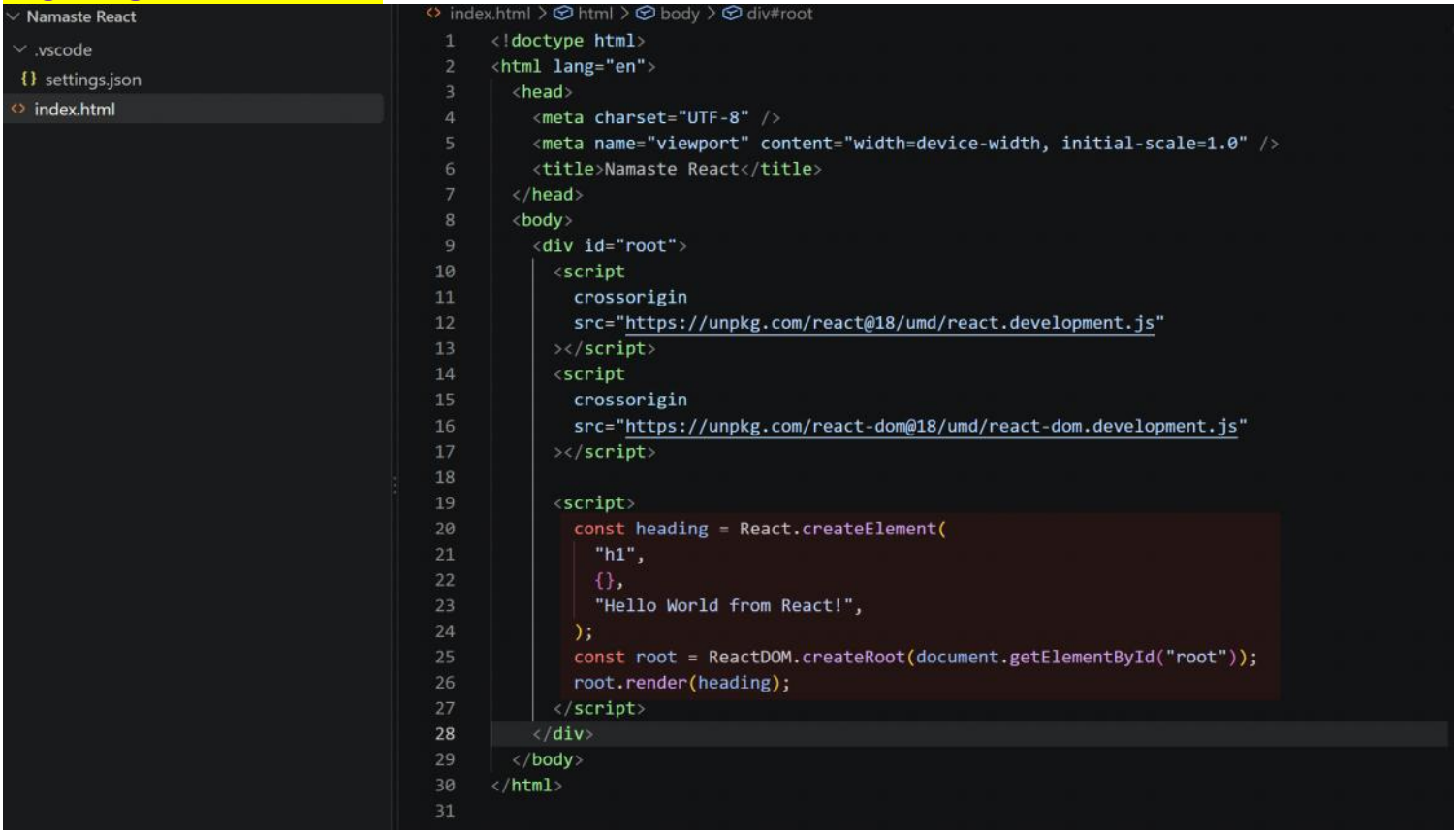
- **CDN = Content Delivery Network**
  It’s a globally distributed network of servers that deliver static assets (like JavaScript libraries, CSS, images, videos) to users from the server closest to them.
- **Why it matters:**
  - **Performance:** Faster load times because files are served from geographically closer servers.
  - **Reliability:** If one server goes down, another can serve the content.
  - **Scalability:** Handles huge traffic spikes without overloading a single origin server.
- **In your React example:**
  You’re pulling React and ReactDOM from **unpkg CDN**. Instead of hosting those files yourself, you rely on unpkg’s distributed servers to deliver them quickly.

What is crossorigin?

- **Definition:** It's an attribute you add to `<script>` or `<link>` tags to control how browsers handle **cross-origin requests** (requests to a domain different from your page's domain).
- **Why React CDN uses it:**
  - When you load React from `https://unpkg.com`, that's a different origin than your local HTML file.
  - `crossorigin` ensures the browser fetches the script in a way that allows proper error reporting and caching.
  - Without it, if the script fails, you might get a generic "Script error" instead of a detailed stack trace.
- **Common values:**
  - `anonymous` → No credentials (cookies, auth headers) are sent. This is the default and safest.
  - `use-credentials` → Credentials are sent, but the server must explicitly allow them via CORS headers.

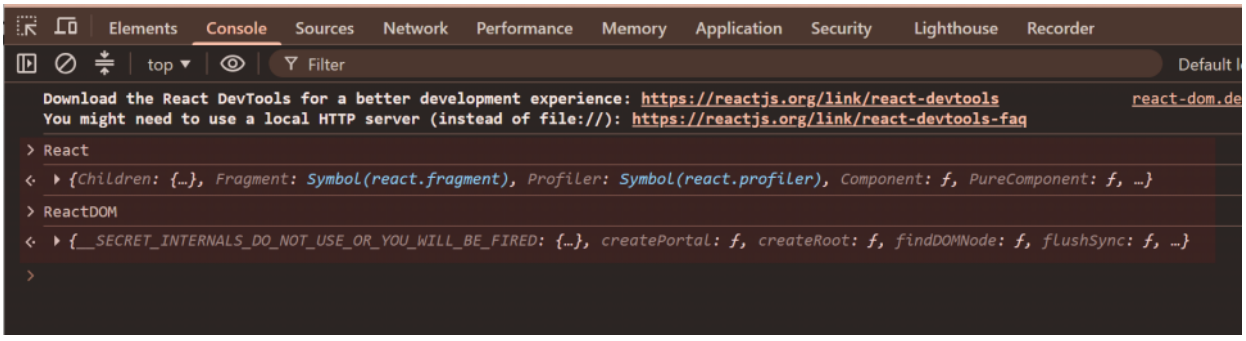
This tells the browser: "Load this script from another origin, but don't send credentials, and allow detailed error reporting."

Beginning with React Code





Hello World from React!




```
const heading = React.createElement("h1", {}, "Hello World from React!");
```

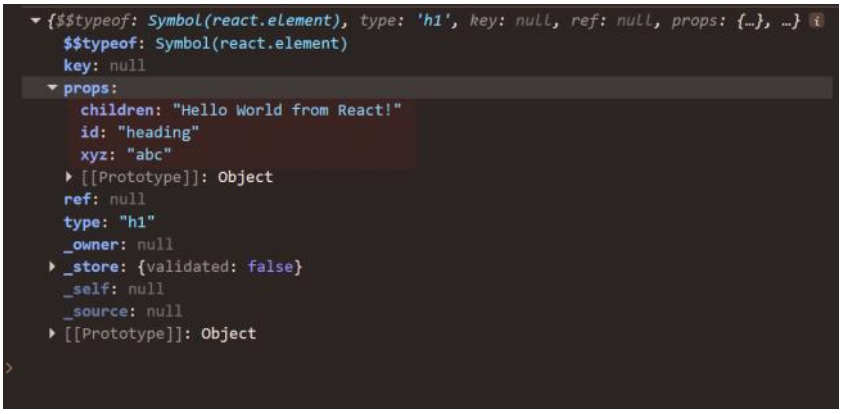
- **What:** This is the raw API — React elements are just plain JS objects describing UI.
- **Why:** It shows how JSX compiles under the hood. JSX is syntactic sugar for `React.createElement`.
- **Senior Note:** We rarely write `createElement` manually. JSX is cleaner, more readable, and integrates better with tooling.

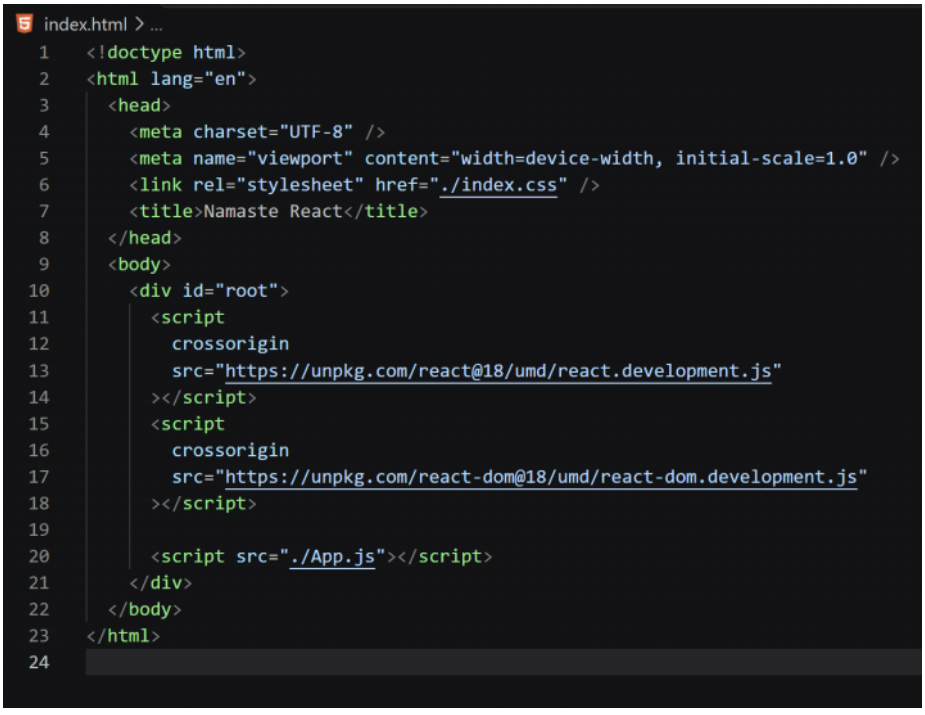
```
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(heading);
```

- **What:** This is the React 18+ way of rendering. It enables **Concurrent Mode** features like automatic batching and better scheduling.
- **Why:** `ReactDOM.render` (old API) is deprecated. Always use `createRoot`.
- **Senior Note:** In real apps, we usually render a **component tree**, not a single element.

JS App.js > ...

```
1  const heading = React.createElement(
2    "h1",
3    {id:"heading", xyz:"abc"},
4    "Hello World from React!",
5  );
6  console.log(heading); //object
7  const root = ReactDOM.createRoot(document.getElementById("root"));
8  root.render(heading);
```





When you call:

Js Copy

```
const element = React.createElement("h1", {}, "Hello World");
```

React doesn't immediately create a DOM node. Instead, it returns a **plain JavaScript object** that looks like:

Js Copy

```
{
  type: "h1",
  props: {
    children: "Hello World"
  }
}
```

This is called a **ReactElement**. It's just a description of what you want on the screen — the **virtual DOM**.

Root.render:-

When you call:

```js
root.render(element);
```

React takes that object and:

- Compares it to the previous virtual DOM (diffing).
- Figures out what changed.
- Updates the **real DOM** efficiently.

So the browser finally sees:

```html
<h1>Hello World</h1>
```

From ReactElement to DOM: Understanding Nested Structures

Desired DOM Structure

```html
<div id="parent">
  <div id="child">
    <h1>I am h1 tag</h1>
    <h2>I am h2 tag</h2>
  </div>
</div>
```

React Code

```js
const parent = React.createElement(
  "div",
  { id: "parent" },
  React.createElement("div", { id: "child" }, [
    React.createElement("h1", {}, "I am h1 tag"),
    React.createElement("h2", {}, "I am h2 tag")
  ])
);
```

```
/**
 *
 * <div id="parent">
 *   <div id="child">
 *     <h1>I am h1 tag</h1>
 *   </div>
 * </div>
 *
 *
 */

const parent = React.createElement(
  "div",
  {id:"parent"},
  React.createElement( "div", {id:"child"},[
    React.createElement("h1", {}, "I am h1 tag"),
    React.createElement("h2", {}, "I am h2 tag")
  ])
);
```

```
<!DOCTYPE html>
<html lang="en"> view-source
 <head> ... </head>
 <body>
  <div id="root">
   <div id="parent">
    <div id="child"> == $0
     <h1>I am h1 tag</h1>
     <h2>I am h2 tag</h2>
    </div>
   </div>
  </div>
 </body>
</html>
```

```
const parent = React.createElement("div", {id:"parent"}, [
  React.createElement( "div", {id:"child"},[
    React.createElement("h1", {}, "I am h1 tag"),
    React.createElement("h2", {}, "I am h2 tag")
  ]),
  React.createElement( "div", {id:"child2"},[
    React.createElement("h1", {}, "I am h1 tag"),
    React.createElement("h2", {}, "I am h2 tag")
  ]),
]);
```

```
<!DOCTYPE html>
<html lang="en"> view-source
 <head> ... </head>
 <body>
  <div id="root">
   <div id="parent">
    <div id="child">
     <h1>I am h1 tag</h1>
     <h2>I am h2 tag</h2>
    </div>
    <div id="child2">
     <h1>I am h1 tag</h1>
     <h2>I am h2 tag</h2> == $0
    </div>
   </div>
  </div>
 </body>
</html>
```

Script Loading Order in React (CDN Setup)

```
<body>
  <div id="root"></div>

  <!-- React core libraries first -->
  <script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></sc
  <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.

  <!-- Your app logic last -->
  <script src="./App.js"></script>
</body>
```

Why This Order Matters

- **React & ReactDOM must load first**
  Your `App.js` depends on these libraries. If `App.js` runs before React is available, you'll get errors like `React is not defined`.
- **App.js last**
  This ensures your custom code executes only after the framework is ready.

Case 1: Root has static HTML

Html

```
<body>
  <div id="root">
    <h1>Static HTML Heading</h1>
    <p>This is plain HTML text</p>
  </div>

  <script src="react.js"></script>
  <script src="react-dom.js"></script>
  <script src="./App.js"></script>
</body>
```

App.js

Js Copy

```
const parent = React.createElement("div", {id:"parent"}, [
  React.createElement("h1", {}, "I am React h1"),
  React.createElement("h2", {}, "I am React h2")
]);

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(parent);
```

Browser Output

Html Copy

```
<div id="parent">
  <h1>I am React h1</h1>
  <h2>I am React h2</h2>
</div>
```

👉 Notice: The original `<h1>` and `<p>` inside `#root` are **gone**. React replaced them with its own tree.

❓ Question

If the `<div id="root">` already contains `<h1>` or other tags in plain HTML, will React append new elements or replace them?

✅ Answer

- **ReactDOM.createRoot(...).render(element)** will **replace** the existing content inside the root container.
- Whatever static HTML you had inside `<div id="root">` will be cleared, and React will take full control of that container.
- React does not "append" to existing DOM nodes — it reconciles its **virtual DOM tree** with the real DOM, starting fresh from the root.

- **Library:** You call the library when you need its functionality.
- **Framework:** The framework calls your code at the appropriate time and controls the application's overall flow.

Library: normal control flow

Your application

↓

Calls library method

Framework: inverted control flow

Framework runtime

↓

Invokes your controller or component

A library is a collection of reusable functionality that developers invoke explicitly to solve specific problems. A framework provides a predefined structure and controls the application's execution lifecycle, allowing developers to extend it through controllers, components, hooks, and other extension points.

The fundamental difference is **Inversion of Control**: with a library, the application calls the library; with a framework, the framework often calls the application code. For example, LINQ is a library that I call from C#, while ASP.NET Core is a framework that invokes my controller actions as part of its HTTP request pipeline.

Igniting our React App

Tuesday, October 13, 2026 5:50 PM

Git Setup for Namaste-React Project :-

1. **Initialize Git in local folder**

```
Bash
git init
```

2. **Rename branch to** `main`

```
Bash
git branch -M main
```

3. **Stage and commit files**

```
Bash
git add .
git commit -m "episode-01"
```

4. **Add remote repository**

```
Bash                                                    Copy
git remote add origin https://github.com/Aparup1503/namaste-react.git
```

5. **First push attempt failed**

- Reason: remote already had a commit (README).
- Fix: fetch and rebase before pushing.

Only if Add remote repository fails:-

6. **Fetch remote branch**

```bash
git fetch origin
```

7. **Rebase local branch with remote**

```bash
git pull --rebase origin main
```

8. **Push successfully**

```bash
git push origin main
```

✅ After this, your local project is connected to GitHub.
👉 Future workflow is simple:

```bash
git add .
git commit -m "your message"
git push
```